Research Note

Efficiently Explaining Constraint Satisfaction Problems with Optimal Unsatisfiable Subsets: Extended use-cases and improved algorithms

Emilio Gamba EMILIO.GAMBA@VUB.BE
Bart Bogaerts BART.BOGAERTS@VUB.BE
Vrije Universiteit Brussel, Pleinlaan 5
1050 Elsene, Belgium

Tias Guns TIAS.GUNS@VUB.BE
KULeuven, Oude Markt 13 - bus 5500,
3000 Leuven, Belgium

Contents

1 Introduction **3**
1.1 Contributions 3

2 Related Works **4**

3 Background **4**

4 Motivation **5**

5 Optimal Constraint Unsatisfiable Subsets **6**

6 Efficient OCUS computation for explanations **8**
6.1 Incremental OCUS Computation. 10
6.2 Domain-Specific Implementations of GROW. 11
6.3 Explanations with Iterated OUS 11
6.4 Correction Subsets enumeration 13
6.5 Interactivity 13

7 Experiments **14**
7.1 Experimental Setup 14
7.2 Explanation quality 14
7.3 Domain specific grow 15
7.4 Improving Correction Subset extraction 16
7.5 Speeding up the start 16
7.6 Constrainedness and Incrementality 16

8 Conclusion and Future work **22**

References **22**

Contributions

Overview

1. Introduction
    - Introduce the problem of generating a Sequence of explanations for explaining the solutions of CSPs.
    - Something about explainability in artificial intelligence: workshops and conferences..
    - unsatisfiability as a way to explain
    - Something as followup work on our constraint satisfaction problems
2. Related works
    - something about explanations and quality of the explanations
    - something about reviewing explanations for csps presented at IJCAI
    - something about weighted Unsatisfiable subsets
    - reviewing improvements that apply to unsatisfiable subsets
        - (?) Abstract cores
        - Disjoint MCS enumeration
        - something about incrementaltity (and more specifically) in maxsat solving which could be applied to weighted MUSes
    - Searching for explainable solutions in Sudoku
3. Background
    - Introduce MUSes/MCSes/Implicit HS ...
4. Motivation
    - time to generate explanations too slow
    - Require optimality
    - Calls to OCUS very similar (incrementality/better search)
5. OCUS
6. Efficient OCUS computation for explanations
7. Experiments
    - RQ1. Cost comparison between MUS and Constrained for Logic Grid Puzzles+ Extra Demystify puzzles?
    - RQ2. Which **domain-specific Grow methods** perform best: MaxSAT vs disj. mcs?

- RQ3. Effect of Constrainedness on the time required to computed explanations?
- RQ4. Does re-use of information across the different iterations improve efficiency?
  - testing OUSb incremental/OCUS incremental vs OUSb/OUS/OCUS
- RQ5. Does appropriate sorting of literals lead to faster generation of individual explanations and explanation sequences:
  - Iterative OUS vs Best configurations
- RQ6. Path towards interactive: Individual explanation time not incremental

8. Conclusion and future work
9. Acknowledgements

1. Introduction

1.1 Contributions

Motivated by these observations, we develop algorithms that aid explaining CSPs and improve the state-of-the-art in the following ways:

- We develop algorithms that compute (cost-)**Optimal** Unsatisfiable Subsets (from now on called OUSs) based on the well-known hitting-set duality that is also used for computing cardinality-minimal MUSs (Ignatiev, Previti, Liffiton, & Marques-Silva, 2015; Saikko, Wallner, & Järvisalo, 2016).
- We observe that many of the individual calls for MUSs (or OUSs) can actually be replaced by a single call that searches for an optimal unsatisfiable subset **among subsets satisfying certain structural constraints**. In other words, we introduce the *Optimal **Constrained** Unsatisfiable Subsets (OCUS)* problem and we show how $O(n^2)$ calls to MUS/OUS can be replaced by $O(n)$ calls to an OCUS oracle, where n denotes the number of facts to explain.
- We develop techniques for **optimizing** the O(C)US algorithms further, exploiting domain-specific information coming from the fact that we are in the *explanation-generation context*. Such optimizations include the development of methods for **information re-use** between consecutive OCUS calls;
- We improve runtime zoals disjoint mcses as an extra step after the grow.
- We introduce an iterative algorithm that takes advantage of the cost of previously computed hitting sets and appropriate sorting on the literals-to-explain to reduce the time to generate an individual explanation as well as the whole explanation sequence.
- We improve the benchmark dataset of the experimental results by testing our approach on a larger number of CSPs and Weighted Unsatisfiable Instances.

Publication history This article is an extension of a previous paper presented at the International Joint Conference on Artificial Intelligence 2021 (Gamba, Bogaerts, & Guns, 2021).

Add what's new:
6.3/6.4/6.5
+ extensive experiments (more puzzles)
-> we evaluate it on a larger set of CSP isntances

- something about explanations and quality of the explanations
- something about reviewing explanations for csps presented at IJCAI
- something about weighted Unsatisfiable subsets
- reviewing improvements that apply to unsatisfiable subsets
  - ~~(?) Abstract cores~~
  - Disjoint MCS enumeration
  - something about incrementaltity (and more specifically) in maxsat solving which could be applied to weighted MUSes
- ~~Searching for explainable solutions in Sudoku~~

in XAI -> CP

Add related works from ijcai
1. implicit hitting set techniques in MaxSAT/MUS
2. check links later in text

(Björnsson, Helgason, & Pálsson, 2021) take a heuristic-search based approach to generate explanation sequences for Constraint Satisfaction problems with a use case on Sudoku puzzles. In that work, the quality of an explanation sequence is measured by a cost function associated with the strategy used, the number of times it is applied and the number of moves it derives. In (Dev Gupta, Genc, & O'Sullivan, 2021), Dev Gupta et al. survey the important solver-agnostic (Junker, 2001; Ignatiev et al., 2015; Felfernig, Schubert, & Zehentner, 2012) and solver-specific methods for generating explanations in Constraint Satisfaction in an effort to connect diverging research directions (Boolean Satisfiability, model-based diagnosis, constraint programming, quantified logics, and related areas). Emilio: cite some more ? and rephrase

3. Background

Add example, must not be final running example

We present all methods using propositional logic but our results easily generalize to richer languages, such as constraint languages, as long as the semantics is given in terms of a satisfaction relation between expressions in the language and possible states of affairs (assignments of values to variables).

Let Σ be a set of propositional symbols, also called *atoms*; this set is implicit in the rest of the paper. A *literal* is an atom p or its negation $\neg p$. A clause is a disjunction of literals. A formula $\mathcal{F}$ is a conjunction of clauses. Slightly abusing notation, a clause is also viewed as a set of literals and a formula as a set of clauses. We use the term clause and constraint interchangeably.

A (partial) interpretation $\mathcal{I}$ is a consistent (not containing both p and $\neg p$) set of literals, for example $\{x_2, \neg x_3\}$. A *model* of $\mathcal{F}$ is an interpretation that satisfies $\mathcal{F}$; $\mathcal{F}$ is said to be *unsatisfiable* if it has no models. Emilio: Write how satisfaction of a formula is defined Satisfaction of a formula $\mathcal{F}$ by an interpretation is defined as usual (Biere, Heule, van Maaren, & Walsh, 2009).

A literal l is a *consequence* of a formula $\mathcal{F}$ if l holds in all $\mathcal{F}$'s models. In our running example, $I_{end} = \{\neg x_1, x_2, \neg x_3, x_4, \neg x_5\}$ is the *maximal consequence* that holds in all models and is also called the *end interpretation*. If I is a set of literals, we write $\overline{I}$ for the set of literals $\{\neg l \mid l \in I\}$. For example, if we define $I_0 = \emptyset$ as the initial interpretation, then $I_{end} \setminus I_0 = \{\neg x_1, x_2, \neg x_3, x_4, \neg x_5\}$ represents the remaining literals to derive and $\overline{I_{end} \setminus I_0} = \{x_1, \neg x_2, x_3, \neg x_4, x_5\}$ represents the complement of the remaining literals.

In the following, we introduce key properties on unsatisfiable subsets on which we build our algorithms.

Definition 1. *A* Minimal Unsatisfiable Subset *(MUS) of $\mathcal{F}$ is an unsatisfiable subset $\mathcal{S}$ of $\mathcal{F}$ for which every strict subset of $\mathcal{S}$ is satisfiable. MUSs($\mathcal{F}$) denotes the set of MUSs of $\mathcal{F}$.*

Definition 2. *A set $\mathcal{S} \subseteq \mathcal{F}$ is a* Maximal Satisfiable Subset *(MSS) of $\mathcal{F}$ if $\mathcal{S}$ is satisfiable and for all $\mathcal{S}'$ with $\mathcal{S} \subsetneq \mathcal{S}' \subseteq \mathcal{F}$, $\mathcal{S}'$ is unsatisfiable.*

Definition 3. *A set $\mathcal{S} \subseteq \mathcal{F}$ is a* correction subset *of $\mathcal{F}$ if $\mathcal{F} \setminus \mathcal{S}$ is satisfiable. Such a $\mathcal{S}$ is a* minimal correction subset *(MCS) of $\mathcal{F}$ if no strict subset of $\mathcal{S}$ is also a correction subset. MCSs($\mathcal{F}$) denotes the set of MCSs of $\mathcal{F}$.*

Each MCS of $\mathcal{F}$ is the complement of an MSS of $\mathcal{F}$ and vice versa.

Definition 4. *Given a collection of sets $\mathcal{H}$, a hitting set of $\mathcal{H}$ is a set h such that $h \cap C \neq \emptyset$ for every $C \in \mathcal{H}$. A hitting set is* minimal *if no strict subset of it is also a hitting set.*

The next proposition is the well-known hitting set duality (Liffiton & Sakallah, 2008; Reiter, 1987) between MCSs and MUSs that forms the basis of our algorithms, as well as algorithms to compute MSSs (Davies & Bacchus, 2013) and *cardinality-minimal* MUSs (Ignatiev et al., 2015).

Proposition 1. *A set $\mathcal{S} \subseteq \mathcal{F}$ is an MCS of $\mathcal{F}$ iff it is a* minimal hitting set *of MUSs($\mathcal{F}$). A set $\mathcal{S} \subseteq \mathcal{F}$ is a MUS of $\mathcal{F}$ iff it is a* minimal hitting set *of MCSs($\mathcal{F}$).*

4. Motivation

(running) Consider these four clauses over the different sectiosn

Our work is motivated by the problem of explaining satisfaction problems through a sequence of simple explanation steps. This can be used to teach people problem-solving skills, to compare the difficulty of related satisfaction problems (through the number and complexity of steps needed), and in human-computer solving assistants.

Our original explanation generation algorithm (Bogaerts, Gamba, Claes, & Guns, 2020) starts from a formula $\mathcal{C}$ (in the application coming from a high level CSP), a partial interpretation I (here also viewed as a conjunction of literals) and a cost function f quantifying the difficulty of an explanation step, by means of a weight for every clause and literal in $\mathcal{F}$.

Algorithm 1: EXPLAIN-ONE-STEP($\mathcal{C}, f, I, I_{end}$)

```
1 X_best ← nil
2 for l ∈ {I_end \ I} do
3 |   X ← MUS(C ∧ I ∧ ¬l)
4 |   if f(X) < f(X_best) then
5 |   |   X_best ← X
6 |   end
7 end
8 return X_best
```

X -> S

The goal is to find a sequence of *simple* explanation steps, where the simplicity of a step is measured by the total cost of the elements used in the explanation. An explanation step is an implication $I' \wedge \mathcal{C}' \implies N$ where I' is a subset of already derived literals, $\mathcal{C}'$ is a subset of constraints of the input formula $\mathcal{C}$, and N is a set of literals entailed by I' and $\mathcal{C}'$ which are not yet explained.

The key part of the algorithm is the search for the next best explanation, given an interpretation I derived so far. Algorithm 1 shows the gist of how this was done. It takes as input the formula $\mathcal{C}$, a cost function f quantifying the quality of explanations, an interpretation I containing all already derived literals in the sequence so far, and the interpretation-to-explain I_{end}. To compute an explanation, this procedure iterates over the literals that are still to explain, computes for each of them an associated MUS and subsequently selects the lowest cost one from found MUSs. The reason this works is because

there is a one-to-one correspondence between MUSs of $\mathcal{C} \wedge I \wedge \neg l$ and so-called *non-redundant explanation* of l in terms of (subsets of) $\mathcal{C}$ and I (Bogaerts et al., 2020).

Experiments have shown that such a MUS-based approach can easily take hours, especially when multiple MUS calls are performed to increase the chance of finding a good MUS, and hence that algorithmic improvements are needed to make it more practical (Bogaerts, Gamba, & Guns, 2021). We see three main points of improvement, all of which will be tackled by our generic OCUS algorithm presented in the next section.

- First of all, since the algorithm is based on MUS calls, there is no guarantee that the explanation found is indeed optimal (with respect to the given cost function). Performing multiple MUS calls is only a heuristic that is used to circumvent the restriction that *there are no algorithms for cost-based unsatisfiable subset* ***optimization***.
- Second, this algorithm uses MUS calls for every literal to explain separately. The goal of all these calls is to find a single unsatisfiable subset of $\mathcal{C} \wedge I \wedge \overline{(I_{end} \setminus I)}$ that contains exactly one literal from $\overline{(I_{end} \setminus I)}$. This begs the questions whether it is possible *to compute a single (optimal) unsatisfiable subset* ***subject to constraints***, where in our case, the constraint is to include exactly one literal from $\overline{(I_{end} \setminus I)}$.
- Finally, the algorithm that computes an entire explanation sequence makes use of repeated calls to Explain-One-Step and hence will solve many similar problems. This raises the issue of ***incrementality****: can we re-use the computed data structures to achieve speed-ups in later calls?*

5. Optimal Constraint Unsatisfiable Subsets

Example extend

The first two considerations from the previous section lead to the following definition.

Definition 5. *Let $\mathcal{F}$ be a formula, $f : 2^{\mathcal{F}} \to \mathbb{N}$ a cost function and p a predicate $p : 2^{\mathcal{F}} \to \{true, false\}$. We call $\mathcal{S} \subseteq \mathcal{F}$ an OCUS of $\mathcal{F}$ (with respect to f and p) if*

- *$\mathcal{S}$ is unsatisfiable,*
- *$p(\mathcal{S})$ is true*
- *all other unsatisfiable $\mathcal{S}' \subseteq \mathcal{F}$ for which $p(\mathcal{S}')$ is true satisfy $f(\mathcal{S}') \geq f(\mathcal{S})$.*

If we assume that the predicate p is specified itself as a CNF over (meta)-variables indicating inclusion of clauses of $\mathcal{F}$, and f is obtained by assigning a weight to each such meta-variable, then the complexity of the problem of finding an OCUS is the same as that of the SMUS (cardinality-minimal MUS) problem (Ignatiev et al., 2015): the associated decision problem is Σ_2^P-complete. Hardness follows from the fact that SMUS is a special case of OCUS, containment follows - intuitively - from the fact that this can be encoded as an $\exists\forall$-QBF using a Boolean circuit encoding of the costs.

When considering the procedure Explain-One-Step from the perspective of OCUS defined above, the task of the procedure is to compute an OCUS of the formula $\mathcal{F} := \mathcal{C} \wedge I \wedge \overline{I_{end} \setminus I}$ with p the predicate that holds for subsets that contain exactly one literal of $\overline{I_{end} \setminus I}$, see Algorithm 2.

In order to compute an OCUS of a given formula, we propose to build on the hitting set duality of Proposition 1. For this, we will assume to have access to a solver CondOptHittingSet that can compute hitting sets of a given collection of sets that are *optimal* (w.r.t. a given cost function f) among all hitting sets *satisfying a condition* p. The choice of the underlying hitting set solver will thus determine which types of cost functions and constraints are possible. In our implementation, we use a cost function f as well as a condition p that can easily be encoded as linear constraints, thus allowing the use of highly optimized mixed integer programming (MIP) solvers. The CondOptHittingSet formulation is as follows:

$$\begin{aligned} minimize_S \quad & f(S) & \\ s.t. \quad & p(S) & \\ & sum(H) \geq 1, & \forall H \in \mathcal{H} \\ & s \in \{0,1\}, & \forall s \in S \end{aligned}$$

where S is a set of MIP decision variables, one for every clause in $\mathcal{F}$. In our case, p is expressed as $\sum_{s \in \overline{I_{end} \setminus I}} s = 1$. f is a weighted sum over the variables in S. For example, (unit) clauses representing previously derived facts can be given small weights and regular constraints can be given large weights, such that explanations are penalized for including constraints when previously derived facts can be used instead.

Algorithm 2: Explain-One-Step-ocus$(\mathcal{C}, f, I, I_{end})$

```
1 p ← exactly one of the literals in the complement of (I_end \ I)
2 return OCUS(C ∧ I ∧ complement of (I_end \ I), f, p)
```

Algorithm 3: OCUS$(\mathcal{F}, f, p)$

```
1 H ← ∅
2 while true do
3 |   S ← CondOptHittingSet(H, f, p)
4 |   if ¬SAT(S) then
5 |   |   return S
6 |   end
7 |   K ← CorrSubsets(F, S)
8 |   H ← H ∪ K
9 end
```

S <- Grow(F, S)
H <- H u (F \ S)

Algorithm 4: CorrSubsets$(\mathcal{F}, \mathcal{S})$

```
1 return {F \ Grow(F, S)}
```

Our generic algorithm for computing OCUSs is depicted in Algorithm 3. It combines the hitting set-based approach for MUSs of (Ignatiev et al., 2015) with the use of a MIP

solver for (weighted) hitting sets as proposed for maximum satisfiability (Davies & Bacchus, 2013). The key novelty is the ability to add structural constraints to the hitting set solver, without impacting the duality principles of Proposition 1, as we will show.

Ignoring **??** for a moment, the algorithm alternates calls to a hitting set solver with calls to a SAT oracle on a subset $\mathcal{S}$ of $\mathcal{F}$. In case the SAT oracle returns true, i.e., the subset $\mathcal{S}$ is satisfiable, the complement of $\mathcal{S}$ is a correction subset of $\mathcal{F}$ and is added to $\mathcal{H}$.

As in the SMUS algorithm of Ignatiev et al. [(2015)], our algorithm contains an (optional) call to GROW. The purpose of the GROW is to expand a satisfiable subset of $\mathcal{F}$ further, to find a smaller correction subset and as such find stronger constraints on the hitting sets. In our case, the calls for hitting sets will also take into account the cost (f), as well as the meta-level constraints (p); as such, it is not clear a priori which properties a good GROW function should have here. We discuss the different possible implementations of GROW later and evaluate their performance in Section 7. For correctness of the algorithm, all we need to know is that it returns a satisfiable subset $\mathcal{S}'$ of $\mathcal{F}$ with $\mathcal{S} \subseteq \mathcal{S}'$.

Soundness and completeness of the proposal follow from the fact that all sets added to $\mathcal{H}$ are correction subsets, and Theorem 6, which states that what is returned is indeed a solution and that a solution will be found if it exists.

Theorem 6. *Let $\mathcal{H}$ be a set of correction subsets of $\mathcal{F}$. If $\mathcal{S}$ is a hitting set of $\mathcal{H}$ that is f-optimal among the hitting sets of $\mathcal{H}$ satisfying a predicate p, and $\mathcal{S}$ is unsatisfiable, then $\mathcal{S}$ is an OCUS of $\mathcal{F}$. If $\mathcal{H}$ has no hitting sets satisfying p, then $\mathcal{F}$ has no OCUSs.*

Proof. For the first claim, it is clear that $\mathcal{S}$ is unsatisfiable and satisfies p. Hence all we need to show is f-optimality of $\mathcal{S}$. If there would exist some other unsatisfiable subset $\mathcal{S}'$ that satisfies p with $f(\mathcal{S}') \leq f(\mathcal{S})$, we know that $\mathcal{S}'$ would hit every minimal correction set of $\mathcal{F}$, and hence also every set in $\mathcal{H}$ (since every correction set is the superset of a minimal correction set). Since $\mathcal{S}$ is f-optimal among hitting sets of $\mathcal{H}$ satisfying p and $\mathcal{S}'$ also hits $\mathcal{H}$ and satisfies p, it must thus be that $f(\mathcal{S}) = f(\mathcal{S}')$.

The second claim immediately follows from Proposition 1 and the fact that an OCUS is an unsatisfiable subset of $\mathcal{F}$. □

Perhaps surprisingly, correctness of the proposed algorithm does *not* depend on monotonicity properties of f nor p. In principle, any (computable) cost function and condition on the unsatisfiable subsets can be used. In practice however, one is bound by limitations of the chosen hitting set solver.

6. Efficient OCUS computation for explanations

Algorithm 3 is generic and can also be used to find (unconstrained) OUSs, namely with a trivially true p. However, its constrainedness property allows to remove the need to compute a MUS/OUS for every literal. This decreases the complexity of explanation sequence generation from $O(n^2)$ calls to MUS to $O(n)$ calls to OCUS, namely, once for every step in the sequence.

The following example illustrates the optimizations to the OCUS algorithm presented in subsequent sections.

Example 1. *Let $\mathcal{C}$ be a CNF formula over variables x_1, x_2, x_3 with the following four clauses:*

$$c_1 := \neg x_1 \vee \neg x_2 \vee x_3 \qquad c_2 := \neg x_1 \vee x_2 \vee x_3$$

$$c_3 := x_1 \qquad c_4 := \neg x_2 \vee \neg x_3$$

- *I_{end}, the final interpretation, is $\{x_1, \neg x_2, x_3\}$*
- *$\mathcal{I}$, the current interpretation, is $\{\emptyset\}$*
- *$\overline{I_{end} \setminus I} = \{\neg x_1, x_2, \neg x_3\}$.*

To define the input for the OCUS call, we add new clauses representing the final interpretation I_{end}:

$$c_5 := \{x_1\} \qquad c_6 := \{\neg x_2\} \qquad c_7 := \{x_3\}$$

and the complement of the to-be-derived facts $\overline{I_{end} \setminus I}$:

$$c_8 := \{\neg x_1\} \qquad c_9 := \{x_2\} \qquad c_{10} := \{\neg x_3\}$$

The formula $\mathcal{F}$ in the OCUS*-call is thus:*

$$\mathcal{F}_{incr,constr} = \mathcal{C} \wedge I_{end} \wedge \overline{(I_{end} \setminus I)} = \{c_1 \wedge ... \wedge c_{10}\}$$

We define $p \triangleq$ exactly-one(c_8, c_9, c_{10}) and $f = \sum w_i c_i$ with clause weights

$$w_1 = 60, w_2 = 60, w_3 = 100, w_4 = 100, w_5 = \infty, w_6 = \infty, w_7 = \infty, w_8 = 1, w_9 = 1, w_{10} = 1$$

Finally, for explaining one of the literals, we proceed to call OCUS *on formula $\mathcal{F}_{incr,constr}$ with our given cost function f and exactly-one constraint p:*

$$\text{OCUS}(\mathcal{C} \wedge I_{end} \wedge \overline{(I_{end} \setminus I)}, f, p)$$

Table 1 depicts the steps of algorithm 3 for generating an OCUS of given $\mathcal{F}$, f and p. In this example, the GROW we used is the one called *Max-Actual-Unif* in 7.3.

$\mathcal{H}$ is initialized as the empty set. At each iteration, the hitting set solver will search for a cost-minimal assignment that hits all sets in $\mathcal{H}$ and that furthermore contains exactly one of $\{\neg x_1, x_2, \neg x_3\}$ (due to p). Once an unsatisfiable subset is found, it's guaranteed to validate the given constraint and be optimal. In this example, step 6 of Table 1 signifies that using constraint c_3 we can derive x_1, and more formally $c_3 \implies x_1$.

We now discuss optimizations to the OCUS algorithm that are specific to explanation sequence generation, though they can also be used when other forms of domain knowledge are present.

	$\mathcal{S}$	SAT($\mathcal{S}$)	GROW ($\mathcal{S}$, $\mathcal{F}$)	$\mathcal{H} \leftarrow \mathcal{H} \cup \{\mathcal{F} \setminus \mathcal{S}\}$
1	$\{\neg x_1\}$	*true*	$\{c_1, c_2, c_4, \neg x_1, \neg x_2, x_3\}$	$\{\{c_3, x_1, x_2, \neg x_3\}\}$
2	$\{\neg x_3\}$	*true*	$\{c_1, c_2, c_4, \neg x_1, \neg x_2, \neg x_3\}$	$\{..., \{c_3, x_1, x_2, x_3\}\}$
3	$\{x_2\}$	*true*	$\{c_1, c_2, c_3, x_1, x_2, x_3\}$	$\{..., \{c_4, \neg x_1, \neg x_2, \neg x_3\}\}$
4	$\{x_2, c_4\}$	*true*	$\{c_1, c_2, c_4, \neg x_1, x_2, \neg x_3\}$	$\{..., \{x_1, \neg x_2, x_3, c_3\}\}$
5	$\{\neg x_3, c_3\}$	*true*	$\{c_1, c_3, c_4, x_1, \neg x_2, \neg x_3\}$	$\{..., \{c_2, \neg x_1, x_2, x_3\}\}$
6	$\{\neg x_1, c_3\}$	*false*	-	-

Table 1: Sequence of explanation generated with EXPLAIN-ONE-STEP-OCUS

6.1 Incremental OCUS Computation.

Inherently, generating a sequence of explanations still requires as many OCUS calls as there are literals to explain. Indeed, a greedy sequence construction algorithm calls EXPLAIN-ONE-STEP-OCUS iteratively with a growing interpretation I until $I = I_{end}$.

All of these calls to EXPLAIN-ONE-STEP-OCUS , and hence OCUS, are done with very similar input (the set of constraints does not change, and the I slowly grows between two calls). For this reason, it makes sense that information computed during one of the earlier stages can be useful in later stages as well.

The main question is, suppose two OCUS calls are done, first with inputs $\mathcal{F}_1$, f_1, and p_1, and later with $\mathcal{F}_2$, f_2, and p_2; how can we make use as much as possible of the data computations of the first call to speed-up the second call? The answer is surprisingly elegant. The most important data OCUS keeps track of is the collection $\mathcal{H}$ of correction subsets that need to be hit.

This collection in itself is not useful for transfer between two calls, since – unless we assume that $\mathcal{F}_2$ is a subset of $\mathcal{F}_1$, there is no reason to assume that a set in $\mathcal{H}_1$ should also be hit in the second call. However, each set H in $\mathcal{H}$ is the complement (with respect to the formula at hand) of a *satisfiable subset* of constraints, and this satisfiability remains true. Thus, instead of storing $\mathcal{H}$, we can keep track of a set **SSs** of *satisfiable subsets* (the sets $\mathcal{S}$ in the OCUS algorithm). When a second call to OCUS is performed, we can then initialize $\mathcal{H}$ as the complement of each of these satisfiable subsets with respect to $\mathcal{F}_2$, i.e.,

$$\mathcal{H} \leftarrow \{\mathcal{F}_2 \setminus \mathcal{S} \mid \mathcal{S} \in \mathbf{SSs}\}.$$

The effect of this is that we *bootstrap* the hitting set solver with an initial set $\mathcal{H}$.

For hitting set solvers that natively implement incrementality, we can generalize this idea further: we know that all calls to OCUS($\mathcal{F}, f, p$) will be cast with $\mathcal{F} \subseteq \mathcal{C} \cup I_{end} \cup \overline{I_{end} \setminus I_0}$, where I_0 is the start interpretation. Since our implementation uses a MIP solver for computing hitting sets (see Section **??**), and we have this upper bound on the set of formulas to be used, we can initialize all relevant decision variables once. To compute the conditional hitting set for a specific $\mathcal{C} \cup I \cup \overline{I_{end} \setminus I} \subseteq \mathcal{C} \cup I_{end} \cup \overline{I_{end} \setminus I_0}$ we need to ensure that the MIP solver only uses literals in $\mathcal{C} \cup I \cup \overline{I_{end} \setminus I}$, for example by giving all other literals infinite weight in the cost function. In this way, the MIP solver will automatically maintain and reuse previously found sets-to-hit in each of its computations.

uitbreiding met link naar het voorbeeld met/zonder incremnetal:
-> Sat subsets

adapt to correction subset

correction subsets

6.2 Domain-Specific Implementations of Grow.

The goal of the GROW procedure is to turn $\mathcal{S}$ into a larger subformula of $\mathcal{F}$. The effect of this is that the complement added to $\mathcal{H}$ will be smaller, and hence, a stronger restriction for the hitting set solver is found.

Choosing an effective GROW procedure requires finding a difficult balance: on the one hand, we want our subformula to be as large as possible (which ultimately would correspond to computing the maximum satisfiable subformula), but on the other hand we also want the procedure to be very efficient as it is called in every iteration.

For the case of explanations we are in, we make the following observations:

- Our formula at hand (using the notation from the EXPLAIN-ONE-STEP-OCUS algorithm) consists of three types of clauses: 1. (translations of) the problem constraints (this is $\mathcal{C}$) 2. literals representing the assignment found thus far (this is I), and 3. the negations of literals not-yet-derived (this is $\overline{I_{end} \setminus I}$).
- $\mathcal{C}$ and I together are satisfiable, with assignment I_{end}, and *mutually supportive*, by this we mean that making more constraints in $\mathcal{C}$ true, more literals in I will automatically become true and vice versa.
- The constraint p enforces that each hitting set will contain **exactly** one literal of $\overline{I_{end} \setminus I}$

Since the restriction on the third type of elements of $\mathcal{F}$ are already strong, it makes sense to use the GROW $(\mathcal{S},\mathcal{F})$ procedure to search for a *maximal* satisfiable subset of $\mathcal{C} \cup I$ with hard constraints that $\mathcal{S}$ should be satisfied, using a call to an efficient (partial) MaxSAT solver. Furthermore, we can initialize this call as well as any call to a `SAT` solver with the polarities for all variables set to the value they take in I_{end}.

We evaluate different grow strategies in the experiments section partial MaxSAT as well as weighted partial MaxSAT based on th function f.

Algorithm 1: EXPLAIN-ONE-STEP$(\mathcal{C}, f, I, I_{end})$

```
1 X_best ← nil
2 for l ∈ {I_end \ I} do
3 |   X ← MUS(C ∧ I ∧ ¬l)
4 |   if f(X) < f(X_best) then
5 |   |   X_best ← X
6 |   end
7 end
8 return X_best
```

X -> S

To new section (bounding + iterative)

6.3 Explanations with Iterated OUS

Instead of working OCUS-based, we can also generate optimal exp the MUS call by an OUS call in Algorithm 1 (where OUS is compu but with a trivially true p). When doing this, we know that every OCUS of cost greater than or equal to $f(X_{best})$ will be discarded by the check on Line 4 of Algorithm 1.

Ous ->boundedOUS

Bounded OUS As such, a next optimization is to, instead of searching for an OUS, perform a *bounded OUS check*, which only computes an OUS in case one of cost smaller than a given bound ub exists. In our specific implementation, bounded OUS is performed by interrupting this OUS-call (after Line 3 Algorithm 3) if $f(\mathcal{S}) > ub$.

Emilio: *Rephrase and incorporate with previous:* Since the bounding on the OUS cost has the most effect if cheap OUSs are found early in the loop across the different literals, we keep track of an upper bound of the cost of an OUS for each literal to explain. This is initialized to a value greater than any OUS, e.g., as $f(\mathcal{C} \wedge I_0 \wedge \overline{I_{end} \setminus I_0})$, and is updated every time an OUS explaining that literal is found; when going through the loop in Line 2 of Algorithm 3, we then handle literals in order of increasing upper bounds.

Incremental algorithm presentation
Adding Bounded + Sorting -> onestep as well
for-loop

Algorithm gradually construct it to algorithm 5
- OUS-next-candidate S, optimal?

Iterated OUS Explanations with bounded OUSs still computes extra hitting sets until the upper bound is reached or until a OUS is found. Therefore, instead of sorting the literals by the OUSs that are already found, we go one step further and keep track of the already found hitting sets as a lower bound on the cost of the OUS. More precisely, Algorithm 5 keeps track of a priority queue sorted by the cost of the last computed hitting set for every remaining literal to explain.

First, line 3 through 8 initializes, for every remaining literal to explain $\ell \in (I_{end} \setminus I)$, the priority queue with tuple $\langle f(\mathcal{S}), \ell, \mathcal{S}\rangle$ where $\mathcal{S}$ is the last computed hitting set for ℓ on the collection of sets-to-hit $\mathcal{H}_\ell$; and its correspond cost $f(\mathcal{S})$.

Next, line 10 extracts the best candidate literal-to-explain out of the priority queue. Similar to Algorithm 3, if corresponding hitting set is unsatisfiable, it is guaranteed to be an OCUS, otherwise a number of correction subsets are extracted from $\mathcal{S}$ and added to the collection of sets to hit $\mathcal{H}_\ell$. Finally, a new hitting set is computed and the literal is pushed back in the priority queue.

Emilio: Idea: Generate a table with number of calls to optimal hitting set in order to compare the speedup of using iterated vs bounded optimized

Algorithm 5: EXPLAIN-ONE-STEP-ITERATIVE$(\mathcal{C}, f, I, I_{end})$

```
   // Initilisation of priority Queue with tuples ⟨cost, literal, hs⟩
 1 lb ← PriorityQueue()
 2 p ← none-of(I_end \ I)
   // Emilio:  I'm not sure 'p ← none-of' is the best way to say that we
      cannot take the not-yet-derived literals , or maybe adapt
      F_ℓ ← C ∧ (I) ∧ {¬ℓ}, but if I do that then we don't take
      incrementality into acount..
 3 foreach ℓ ∈ (I_end \ I) do
 4 |   F_ℓ ← C ∧ (I_end \ ℓ) ∧ {¬ℓ}
 5 |   H_ℓ ← ∅
 6 |   S ← CondOptHittingSet(H_ℓ, f, p)            // Optimal hitting set
 7 |   lb.push(⟨f(S), ℓ, S⟩)
 8 end
   // OCUS loop
 9 while true do
10 |   (cost, ℓ, S) ←lb.pop()                      // Literal with lowest bound
11 |   if ¬SAT(S) then
12 |   |   return S                                // OCUS found
13 |   end
14 |   H_ℓ ← H_ℓ ∪ CorrSubsets(F_ℓ, S)
15 |   S ← CondOptHittingSet(H_ℓ, f, p)
16 |   lb.push(⟨f(S), ℓ, S⟩)
17 end
```

Ous-next candidate

EXPLAIN-ONE-STEP-ITERATIVE ensures that only the most promising candidate literals-to-explain are expanded at any given iteration to find the next best explanation.

6.3 Correction Subsets enumeration

Both the OCUS and the EXPLAIN-ONE-STEP-ITERATIVE algorithm rely on repeatedly computing hitting sets and subsequently correction subsets. The question is: can we find non-overlapping correction subsets to impose independent constraints on the sets-to-hit?

Inspired by (Marques-Silva, Heras, Janota, Previti, & Belov, 2013), the CORRSUBSETS procedure, depicted in Algorithm 6, computes a collection of disjoint minimum correction subsets $\mathcal{K}$ starting from a given subset of constraints $\mathcal{S}$ (either the empty set or a computed hitting set). The CORRSUBSETS procedure will repeatedly compute a satisfiable subset $\mathcal{S} \subseteq \mathcal{S}'$ and add its complement to (i) the collection of disjoint MCSes and; (ii) $\mathcal{S}$, ensuring that the constraints of the correction subset cannot be present in the next correction subset, until no more satisfiable subsets can be found.

Algorithm 6: CORRSUBSETS$(\mathcal{F}, \mathcal{S})$

```
1 K, S' ← ∅, S
2 while SAT(S') do
3 |   C ← F \ GROW(S', F)
4 |   S' ← S' ∪ C
5 |   K ← K ∪ {C}
6 end
7 return K
```

6.5 Interactivity

Emilio: Add a note here

- What is an "interactive context"? In text definition
- Did we have to explain the previous steps as well?

6.5 Bootstrapping - schema

Running example ? Uitleg?

1.For l in remaining literala
-> Bootstrapping (correction susbet enumeration)

2. For l in remaining literala
-> for l in remaining literals
OUS (correction susbet enumeration ipv grow)

2. For l in remaining literala
-> for l in remaining literals
. Bootstrapping (correction susbet enumeration)
OUS (correction susbet enumeration ipv grow)

RQ1 = Q1)
RQ2 = Q4) Incrementality on runtime (tabel in 7.5 also for OUS(b))
-> Domain Specific MaxSAT
RQ3 = Which domain-specific correction subset method improves the overal runtime ?
RQ4 = (After clarification in algo) Does startup help improve overal runtime

7. Experiments

Emilio: Needs to be rephrased, parts mostly taken from ijcai paper In this section, we validate the performance improvement of the different versions of our algorithm.

7.1 Experimental Setup

Q1 What is the effect of requiring optimality of the generated MUSs on the **quality** of the generated explanations?

Q2 Which **domain-specific GROW methods** perform best?

Q3 What is the effect of the use of **constrainedness** on the time required to compute an explanation sequence?

Q4 Does **re-use** of information across the different iterations improve efficiency?

This cost function yielded reasonable explanations such that the use of a puzzle-agnostic constraint (considered relatively easy) for all other puzzles we assign a cost of 20 when using a constraint and a cost 1 for facts.

When generating an explanation sequence for these puzzles, the unsatisfiable subset identifies which constraints and which previously derived facts should be combined to derive new information. Our experiments are designed to answer the following research questions:

RQ1 What is the effect on the cost of an explanation when requiring optimal explanations instead of a naive MUS-approach?

correction subset generation

RQ2 Which domain specific grow improve the overall runtime?

merge RQ2 and RQ3

RQ3 How does extraction multiple correction subsets affect runtime (a) as a replacement for grow and its complement; (b) as an initial bootstrapping method?

RQ4 What is the efficiency of a single step O(C)US and is single step sufficiently efficient for an interactive context?

MUS/OUS-bounded/Iterated OUS/OCUS

7.2 Explanation quality

To evaluate the effect of optimality on the quality of the generated explanations, we reimplemented a MUS-based explanation generator based on Algorithm 1. Before presenting the results, we want to stress that this is not a fair comparison with the implementation of Bogaerts et al. [2020], since there – in order to avoid the quality problems we will illustrate below – an extra inner loop was used that employs even more calls to MUS for a selected set of subsets of C of increasing size. While this yields better explanations, it comes at the expense of computation time, thereby leading to several hours to generate the explanation of a single puzzle.

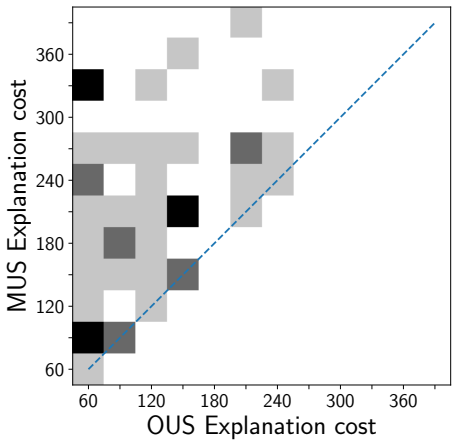


Figure 1: Q1 - Explanation quality comparison of optimal versus subset-minimal explanations in the generated puzzle explanation sequences.

RQ1: clarify the way we generate these costs

To answer Q1, we ran the MUS-based algorithm as described in Algorithm 1 and compared at every step the cost of the produced explanation with the cost of the optimal explanation. These costs are plotted on a heatmap in Figure 1, where the darkness represents the number of occurrences of the combination at hand. We see that the difference in quality is striking in many cases, with the MUS-based solution of- ten missing very cheap explanations (as seen by the two dark squares in the column around cost 60), thereby confirming the need for a cost-based OUS/OCUS approach.

7.3 Domain specific grow

In our OCUS algorithm, we do not just aim to find any satisfiable subsets, but we prefer *high quality* satisfiable subsets: subsets that impose strong constraints on the assignments the optimal hitting set solver can find. This induces a trade-off between *efficiency* of the GROW strategy and *quality* of the produced satisfiable subset.

Thus, to answer **Q2**, we compared variations of OCUS that only differ in which GROW strategy they use. Figure **??** depicts the average (over all the puzzles) cumulative explanation time to derive a number of literals. Note that most puzzles only contain 150 literals, except for 2, which contain 96 and 250 literals respectively. When a method times out for a puzzle at one step, a runtime value of 3600 is used in computing the averages for all future steps. The configurations used are as follows:

- *Max* refers to growing with a MaxSAT solver and *Greedy* to growing using a heuristic method implemented by repeated sat calls, while *no-grow* refers to skipping the GROW step.

- *Full* refers to using the full unsatisfiable formula $\mathcal{F}$ while *Actual* refers to using only the constraints that hold in the final interpretation (see Section **??**). For instance, for the MaxSAT-based calls, *Actual* means that only the previously derived facts and the original constraints are taken into account when computing optimality.

- The MaxSAT solver (*Max*) is combined with different weighing schemes: uniform weights (*unif*), cost-function weights (*pos*) (equal to the weights in f), or the inverse of these costs (*inv*) defined as $\max_j(w_j) + 1 - w_i$.

We can observe that not using a grow strategy performs badly, as do weighted MaxSAT grows with costs w_i (-Pos). Greedy grow strategies improve on not using a grow strategy, but not substantially. The two approaches that work best use the domain-specific knowledge of doing a MaxSAT grow on $\mathcal{C} \cup I$, with the unweighted variant the only one that never times out.

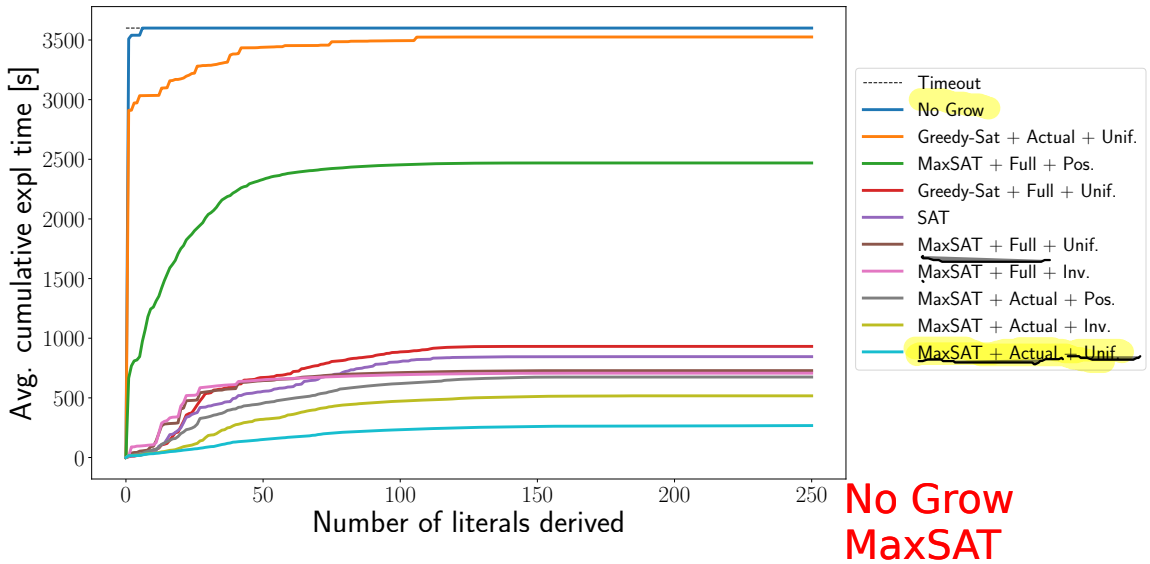


Figure 2: Q2 - Domain Specific grow.

No Grow
MaxSAT
Domain-Specific MaxSAT
Greedy Correction SUbsets
Correction SUbsets

7.4 Improving Correction Subset extraction

Answer RQ3a. Correction Subset extraction as an initial bootstrapping method

7.5 Speeding up the start

Answer RQ3b. Correction Subset extraction as an initial bootstrapping method

config	Time	Opt	Sat	Grow	Remaining	n_{opt}	n_{sat}	n_{grow}
Iter. OUS	29033	7.26% [± 1.1%]	0.78% [± 0.2%]	89.53% [± 1.3%]	2.43% [± 0.4%]	959k	652k	648k
Iter. OUS+Incr.	3910	21.81% [± 4.3%]	0.61% [± 0.1%]	60.02% [± 7.8%]	17.56% [± 3.8%]	365k	58k	54k
OCUS	26992	27.09% [± 7.0%]	0.61% [± 0.1%]	70.4% [± 6.6%]	1.9% [± 0.4%]	474k	474k	470k
OCUS+Incr.	9336	50.36% [± 12.8%]	0.4% [± 0.1%]	36.48% [± 9.2%]	12.76% [± 3.6%]	40k	40k	36k

7.6 Constrainedness and Incrementality

To answer **Q3** and **Q4**, we compare the effect of constrainedness in the search for explanations (C) and incrementality. Next to OCUS, we also include the bounded OUS approach (OUSb), where we call the OUS algorithm for every literal in every step, but we reuse information by giving it the current best bound $f(X_{best})$ and iterating over the literals that performed best in the previous call first. Based on the previous experiment, we always

optie: de grow in 3 figuren

No Grow
MaxSAT
Domain-Specific MaxSAT
Greedy Correction SUbsets
Correction SUbsets

+ OUSb

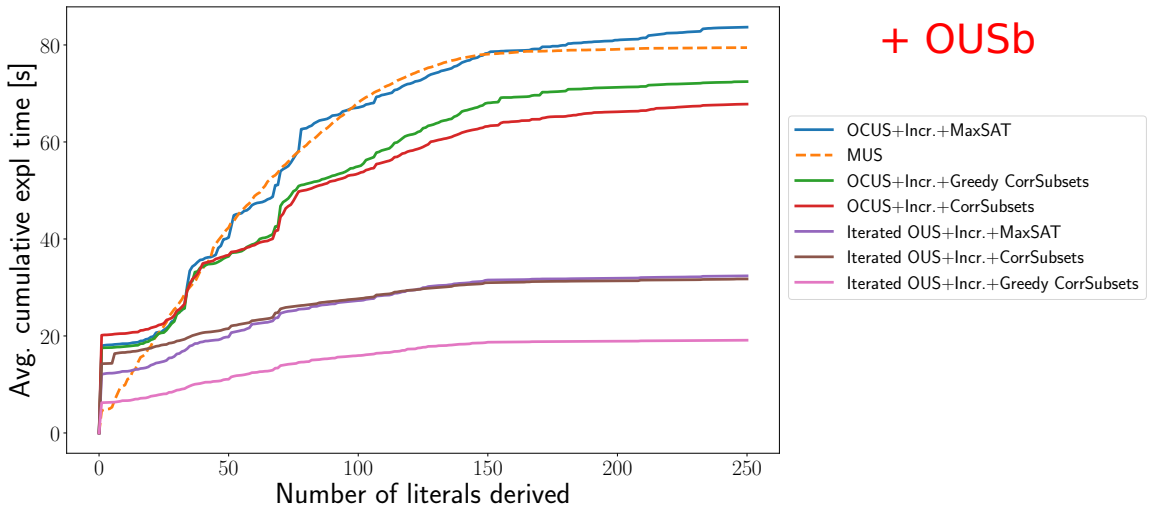


Figure 3: text

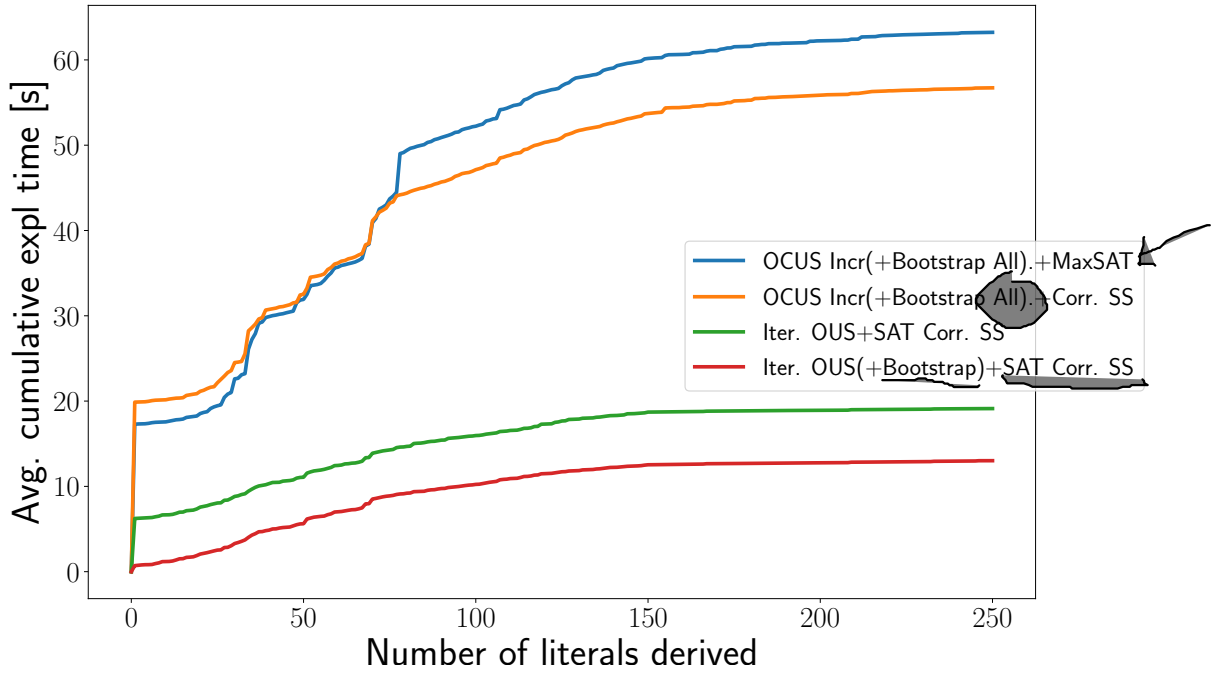


Figure 4: text

use (both for OUSb and OCUS) *Max-Actual-Unif* as grow strategy. For *OCUS*, incrementality (*+Incr. HS*) is achieved by reusing the same incremental MIP hitting set solver throughout the explanation calls, as explained in Section **??**. To have a better view on how incrementality also affects OUSb, we add incrementality to it in the following ways:

- *SS. caching* keeps track of the satisfiable subsets, which are used to initialize $\mathcal{H}$ for a fresh hitting set solver instance each time.

- *Lit. Incr. HS* uses a separate incremental hitting set solver for every literal to explain, throughout the explanation calls. Once the literal is explained, the hitting set solver is discarded.

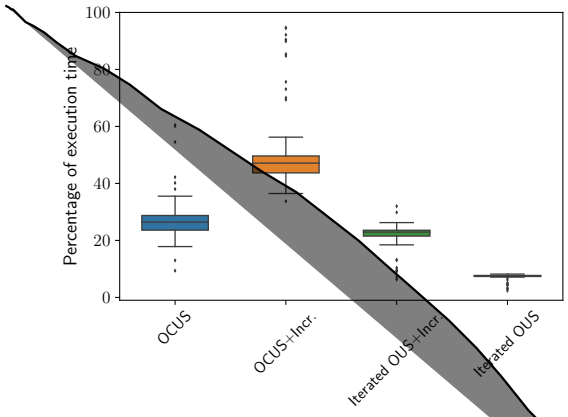
Figure **??** shows the results, where in this figure, the configurations are compared in a similar fashion to Figure **??**.

When comparing the configurations, we see that the plain *MUS*-based implementation is faster than the *O(C)US* implementations, as it solves a simpler problem (with worse quality results as shown in **Q1**). Replacing MUS by bounded OUS calls (orange line) leads to a much larger computation cost. The generic SS caching technique adds additional overhead.

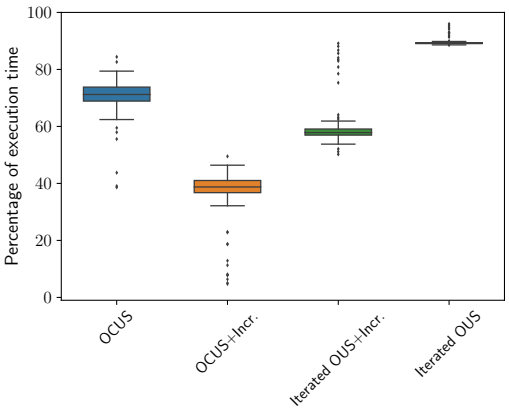
The OCUS variants significantly improve runtime compared to those two bounded OUS approaches, by reducing the number of OUS calls. For OCUS, using an incremental hitting set solver across all steps seems to be slightly faster for deriving literals earlier in the sequence, while inducing a small overhead for the literals at the end of the sequence.

When looking at the runtime to explain the entire sequence, best results are obtained with OUSb + Lit. Incr. HS, that is, using an incremental hitting set solver *for every individual literal to explain*. However, for the first literals, we can see that it takes much more computation time and that reducing the number of OUS calls from n to 1 per explanation step improves runtime (OCUS). However, making each of the n calls incremental and bounded across the entire explanation sequence generation process leads to an even faster process overall (OUSb+Lit. Incr. HS).

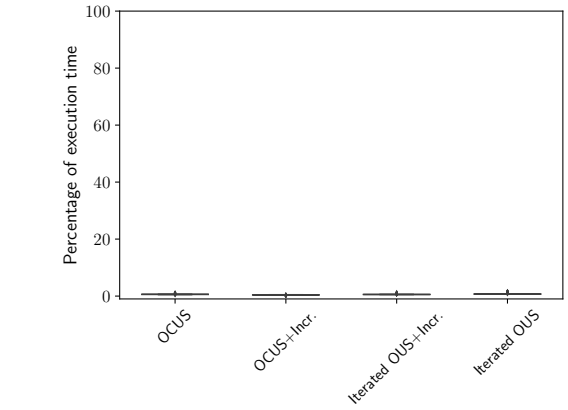
Interactivity Emilio: To analyze how the explanations compare in an interactive context, we represent a distribution of explanation generation times in figure **??**. We see that a major part of the explanations take less than 2 to 5 seconds to be generated. Even though, the difficult explanation steps can take more time ranging from 5 to 100+ seconds to be generated, it is only a small fraction of all generated explanations.



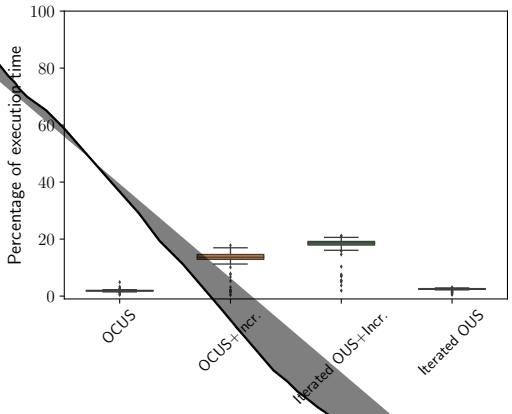
(a) Percentage of the total execution time spent in opt.



(b) Percentage of the total execution time spent in Grow.



(c) Percentage of the total execution time spent in `SAT`.



(d) Percentage of the total execution time spent outside of `SAT`, Grow and Opt.

- Instantanious -> give me an expalnation : Time for first step
- Interactive -> user makes choices
    -> where does the difficult step come from ? Is it the first step or not ?
        -> Most difficult step ? cost explanation?

RQ4. What is the efficiency of a single step O(C)US and how does it compare in an interactive context?

	config	tot_time (s)	avg_t_expl	q_25	q_50	q_75	q_95	q_98	q_100
1	Iter. OUS(+Bootstrap Once)+SAT Corr. SS	1431	0.35	0.12	0.19	0.26	0.91	1.79	54.86
0	Iter. OUS+SAT Corr. SS	2103	0.52	0.13	0.20	0.28	1.47	6.10	51.82
3	OCUS (+Bootstrap All).+Corr. SS	6240	1.56	0.12	0.15	0.48	5.48	21.36	197.72
2	OCUS+Corr. SS	7458	1.86	0.07	0.10	0.65	7.78	22.01	284.55

Figure 6

Emilio: replace the table with correct values from figure

average total time per sequence
average of (average expl time)
boxplot for every quantile over all sequences

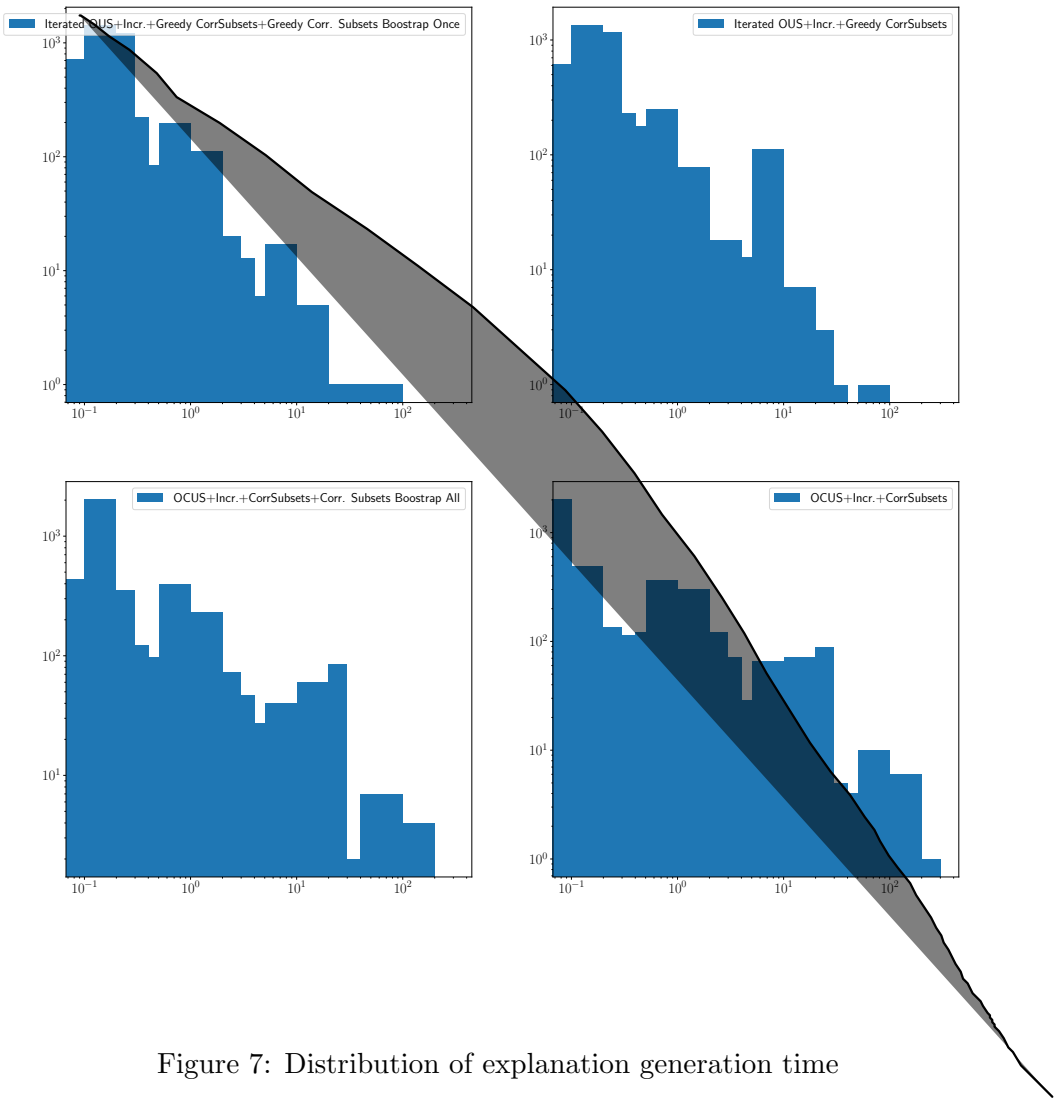


Figure 7: Distribution of explanation generation time

puzzle type	instance	n	avg-n-clauses	avg-lits-to-explain
sudoku	4x4	9	544	41
	9x9	100	14904	501
	16x16	25	154624	2246
	25x25	25	940000	9403
logic	pastaPuzzle	1	4572	96
	150	8	7535	150
	250	1	17577	250
demystify	kakurasu	1	226	16
	thermometer	1	1227	36
	pasta	1	2102	156
	kakuro	6	6961	240
	star-battle	5	7533	82
	nonogram	1	9813	36
	garam	10	14486	756
	origin	1	15919	1620
	binairo	156	16759	99
	skyscrapers	16	17991	159
	tents	4	18644	476
	sudoku	76	34143	729
	miracle	1	59331	729
	x-sums	1	109717	729
	killersudoku	13	3609685	729

Table 2: Characteristics of puzzle instances

8. Conclusion and Future work

Acknowledgments

Appendix A.

References

Biere, A., Heule, M., van Maaren, H., & Walsh, T. (2009). *Handbook of Satisfiability.*

Björnsson, Y., Helgason, S., & Pálsson, A. (2021). Searching for explainable solutions in sudoku. In *2021 IEEE Conference on Games (CoG)*. IEEE.

Bogaerts, B., Gamba, E., Claes, J., & Guns, T. (2020). Step-wise explanations of constraint satisfaction problems. In *Proceedigns of ECAI*, pp. 640–647.

Bogaerts, B., Gamba, E., & Guns, T. (2021). A framework for step-wise explaining how to solve constraint satisfaction problems. *Artificial Intelligence*, *300*, 103550.

Davies, J., & Bacchus, F. (2013). Exploiting the power of MIP solvers in MAXsat. In *Proceedings of SAT*, pp. 166–181.

Dev Gupta, S., Genc, B., & O'Sullivan, B. (2021). Explanation in constraint satisfaction: A survey. In Zhou, Z.-H. (Ed.), *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI-21*, pp. 4400–4407. International Joint Conferences on Artificial Intelligence Organization. Survey Track.

Felfernig, A., Schubert, M., & Zehentner, C. (2012). An efficient diagnosis algorithm for inconsistent constraint sets. *AI EDAM*, *26*(1), 53–62.

Gamba, E., Bogaerts, B., & Guns, T. (2021). Efficiently explaining csps with unsatisfiable subset optimization. In Zhou, Z.-H. (Ed.), *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI-21*, pp. 1381–1388. International Joint Conferences on Artificial Intelligence Organization. Main Track.

Ignatiev, A., Previti, A., Liffiton, M., & Marques-Silva, J. (2015). Smallest MUS extraction with minimal hitting set dualization. In *Proceedings of CP*.

Junker, U. (2001). QuickXPlain: Conflict detection for arbitrary constraint propagation algorithms. In *IJCAI'01 Workshop on Modelling and Solving problems with constraints*.

Liffiton, M. H., & Sakallah, K. A. (2008). Algorithms for computing minimal unsatisfiable subsets of constraints. *J. Autom. Reasoning*, *40*(1), 1–33.

Marques-Silva, J., Heras, F., Janota, M., Previti, A., & Belov, A. (2013). On computing minimal correction subsets. In *Twenty-Third International Joint Conference on Artificial Intelligence*.

Reiter, R. (1987). A theory of diagnosis from first principles. *AIJ*, *32*(1), 57–95.

Saikko, P., Wallner, J. P., & Järvisalo, M. (2016). Implicit hitting set algorithms for reasoning beyond NP. In *Proceedings of KR*, pp. 104–113.